

MASTÈRE 2 – EXPERT IT DEV

Pilotage de projet – M2

MÉMOIRE DE FIN D'ÉTUDES

Projet : IRIS Bank – Site web bancaire

Mastère 2 Expert IT Dev

Sommaire

1. Introduction

2. Présentation de l'entreprise et du contexte

3. Description de la problématique et objectifs

4. Revue de la littérature

La revue de la littérature constitue le socle théorique et conceptuel sur lequel repose le projet IRIS Bank. Elle s'articule autour de trois axes complémentaires : la transformation numérique dans le secteur bancaire, les enjeux de la sécurité des applications web, et les méthodologies de gestion de projet adaptées au développement logiciel moderne.

4.1 La transformation numérique dans le secteur bancaire

La digitalisation des services financiers est une tendance de fond qui redéfinit profondément les rapports entre les établissements bancaires et leurs clients. Selon McKinsey & Company (2023), plus de 70 % des interactions bancaires se font désormais via des canaux numériques, reflet d'une adoption massive des services en ligne et des applications mobiles.

Cette mutation s'explique par plusieurs facteurs convergents. D'une part, la montée en puissance des *fintechs* (entreprises technologiques opérant dans le domaine financier) comme N26, Revolut ou Lydia a contraint les banques traditionnelles à accélérer leur transition numérique pour conserver leur compétitivité (Gomber et al., 2017). D'autre part, les attentes des utilisateurs ont évolué : disponibilité 24h/24, rapidité des transactions, accessibilité multi-device sont devenus des standards incontournables.

Les chercheurs Beccalli, Anolli et Stefanelli (2011) ont démontré que l'investissement dans les technologies de l'information par les banques a un impact direct et mesurable sur leur efficacité opérationnelle. Ils concluent que la digitalisation ne se limite pas à la réduction des coûts, mais améliore également la satisfaction client et fidélise les utilisateurs sur le long terme.

Le projet IRIS Bank s'inscrit pleinement dans ce contexte : il propose une application web bancaire qui centralise la gestion de comptes, les virements et la consultation d'historique, répondant ainsi aux usages actuels des clients bancaires digitaux.

4.2 La sécurité des applications bancaires en ligne

La sécurité informatique est un enjeu critique dans le développement d'applications traitant des données financières sensibles. Les travaux académiques et les rapports sectoriels s'accordent à identifier plusieurs catégories de menaces récurrentes.

4.2.1 Les menaces identifiées

L'OWASP (Open Web Application Security Project), référence mondiale en matière de sécurité applicative, publie régulièrement un classement des dix risques les plus critiques pour les applications web (OWASP Top 10, 2021). Parmi ceux particulièrement pertinents pour une application bancaire :

- **L'injection SQL et les failles d'injection** : des entrées utilisateur mal validées peuvent permettre à un attaquant d'exécuter du code malveillant directement dans la base de données.

- **Le Cross-Site Request Forgery (CSRF)** : cette attaque exploite la session authentifiée d'un utilisateur pour effectuer des actions non consenties, comme un virement bancaire frauduleux (Barth et al., 2008).
- **Les attaques par force brute** : des programmes automatisés testent des milliers de combinaisons identifiant/mot de passe jusqu'à trouver les bonnes. La limitation du taux de requêtes (*rate limiting*) est la contre-mesure recommandée.
- **Le phishing** : des pages frauduleuses imitant l'interface bancaire pour dérober les identifiants. L'usage de HTTPS et de politiques CSP (Content Security Policy) réduit considérablement ce risque.

4.2.2 Les solutions et bonnes pratiques

Face à ces menaces, la littérature technique recommande plusieurs mécanismes de protection, désormais standardisés dans l'industrie :

- **Le hachage des mots de passe** avec des algorithmes cryptographiques robustes tels que bcrypt ou Argon2 (recommandé par le NIST, 2020). Un sel cryptographique (*salt*) est ajouté pour contrer les attaques par table arc-en-ciel.
- **L'authentification à double facteur (2FA)** : une couche de sécurité supplémentaire qui réduit significativement les risques de compromission de compte, même en cas de vol de mot de passe.
- **Les tokens CSRF** : des jetons uniques générés côté serveur pour valider l'authenticité de chaque requête critique.
- **La validation des données côté serveur** : tout paramètre entrant (montants, IBAN, email) est contrôlé avant tout traitement en base de données.

Ces pratiques sont directement appliquées dans le projet IRIS Bank, qui implémente notamment la protection CSRF, le rate limiting via Express.js et le hachage des mots de passe avec bcrypt.

4.3 Architecture des applications web modernes

Les applications web bancaires contemporaines reposent le plus souvent sur une architecture en couches distinctes, communément désignée par le modèle *client-serveur* ou, dans sa forme la plus évoluée, une architecture *microservices* (Richardson, 2018).

Dans le cadre du projet IRIS Bank, une architecture MVC (Modèle-Vue-Contrôleur) a été retenue. Cette approche, popularisée par les frameworks comme Express.js côté back-end, permet une séparation nette des responsabilités :

- **Le Modèle** gère l'accès aux données (MySQL) et la logique métier (calcul de solde, vérification IBAN).
- **La Vue** correspond aux interfaces utilisateur développées en HTML, CSS et JavaScript.
- **Le Contrôleur** orchestre les requêtes HTTP via une API REST, reliant les couches Vue et Modèle.

La communication entre le front-end et le back-end s'effectue via des API REST, format de facto pour les services web modernes (Fielding, 2000). Ce choix architectural facilite la maintenance, les tests unitaires et les évolutions futures du projet.

4.4 Les bases de données relationnelles appliquées au domaine bancaire

La gestion des données financières nécessite des systèmes de gestion de bases de données (SGBD) garantissant l'intégrité, la cohérence et la durabilité des informations. Les bases de données relationnelles, et en particulier MySQL, restent la référence dans ce domaine grâce aux propriétés ACID (Atomicité, Cohérence, Isolation, Durabilité) qu'elles garantissent (Gray & Reuter, 1992).

Pour IRIS Bank, le schéma relationnel repose sur trois entités principales : *users* (clients et administrateurs), *comptes_bancaires* (associés à un ou plusieurs utilisateurs) et *transactions* (traçant l'historique de toutes les opérations). Cette modélisation respecte les principes de normalisation (3NF) afin d'éviter les redondances et d'assurer l'intégrité référentielle.

4.5 Synthèse de la revue de la littérature

L'analyse de la littérature met en évidence trois enseignements majeurs pour le projet IRIS Bank :

- **La digitalisation bancaire** est une réalité incontournable qui justifie le développement d'outils performants, accessibles et centrés sur l'utilisateur.
- **La sécurité** doit être intégrée dès la phase de conception (principe du *Security by Design*) et non ajoutée en fin de projet.
- **L'architecture technique** choisie (MVC, API REST, MySQL) est alignée avec les standards actuels de l'industrie et offre une base solide pour les évolutions futures.

Ces éléments théoriques ont directement orienté les choix techniques et organisationnels du projet, décrits dans la section suivante.

5. Présentation de la démarche choisie

Le choix d'une méthodologie de gestion de projet est une décision stratégique qui conditionne l'organisation du travail, la qualité des livrables et la capacité à s'adapter aux imprévus. Pour le projet IRIS Bank, la démarche retenue est la méthode **Agile Scrum**, complétée par un pipeline d'intégration et de déploiement continu (CI/CD).

5.1 Pourquoi la méthode Agile Scrum ?

La méthode Scrum est un cadre de travail Agile formalisé par Ken Schwaber et Jeff Sutherland en 1995 dans le *Scrum Guide*. Elle repose sur des cycles de développement courts et itératifs appelés *sprints* (généralement de 2 à 4 semaines), permettant de livrer régulièrement des fonctionnalités opérationnelles et de s'adapter rapidement aux retours des parties prenantes.

Scrum a été préféré à une approche en cascade (Waterfall) pour plusieurs raisons :

- **Flexibilité face aux changements** : le périmètre fonctionnel d'IRIS Bank a évolué au cours du projet (ajout de types de comptes, règles métier sur les transactions). Scrum permet d'intégrer ces évolutions sans remettre en cause tout le plan de développement.
- **Livraisons régulières** : chaque sprint produit un incrément fonctionnel, facilitant les revues avec l'encadrant pédagogique et les ajustements progressifs.
- **Travail en équipe structuré** : les rôles Scrum (Product Owner, Scrum Master, équipe de développement) clarifient les responsabilités de chaque membre.
- **Transparence et traçabilité** : le backlog, les user stories et le tableau Kanban offrent une visibilité complète sur l'avancement du projet.

5.2 Organisation de l'équipe et rôles Scrum

L'équipe projet IRIS Bank s'est organisée selon les rôles définis par le cadre Scrum :

- **Product Owner** : responsable de la priorisation du backlog et de la liaison avec les parties prenantes (encadrant pédagogique). Il s'assure que les fonctionnalités développées correspondent aux besoins exprimés dans le cahier des charges.
- **Scrum Master** : facilite les cérémonies Scrum (daily stand-up, sprint planning, rétrospective) et lève les obstacles qui freinent l'équipe.
- **Équipe de développement** : pluridisciplinaire, elle prend en charge les développements back-end (Node.js, Express, MySQL) et front-end (HTML, CSS, JavaScript), ainsi que la rédaction du mémoire.

5.3 Les cérémonies et artefacts Scrum mis en œuvre

5.3.1 Le Product Backlog et les User Stories

Le Product Backlog constitue la liste exhaustive et priorisée de toutes les fonctionnalités attendues du projet IRIS Bank. Chaque élément du backlog est formulé sous forme de *user story* (récit utilisateur) suivant le format standardisé : « *En tant que [rôle], je veux [action] afin de [bénéfice]* ». Ces stories sont saisies et gérées dans l'outil **Jira**, permettant d'associer à chaque ticket un statut (To Do, In Progress, Done), une priorité et une estimation de charge.

5.3.2 Les sprints

Le projet a été découpé en sprints de deux semaines. Chaque sprint démarre par une session de *sprint planning* lors de laquelle l'équipe sélectionne les user stories à réaliser, estime leur charge et définit les critères d'acceptation. À l'issue du sprint, une *sprint review* permet de présenter l'incrément produit, suivie d'une *rétrospective* pour identifier les axes d'amélioration du processus.

5.3.3 Le tableau Kanban et la gestion Jira

Un tableau Kanban est utilisé pour visualiser en temps réel le flux de travail. Dans Jira, les colonnes *Backlog* / *To Do* / *In Progress* / *In Review* / *Done* permettent à toute l'équipe de connaître l'état exact de chaque tâche. Les dépendances entre tickets sont également modélisées dans Jira via des liens de type « *is blocked by* », évitant ainsi les situations de blocage non anticipées.

5.4 La roadmap du projet

La roadmap est un outil de planification stratégique qui présente, sur une ligne temporelle, les grandes étapes (*milestones*) du projet. Pour IRIS Bank, la roadmap a été structurée autour des phases suivantes :

- **Phase 1 – Analyse et cadrage** : rédaction du cahier des charges, analyse des besoins, définition du backlog initial.
- **Phase 2 – Conception** : modélisation UML (diagrammes de cas d'utilisation, de séquence, de classes), conception de la base de données (MCD/MLD), création des maquettes UI.
- **Phase 3 – Développement itératif** : sprints de développement back-end (API, logique métier, sécurité) puis front-end (interfaces client et administrateur).
- **Phase 4 – Tests et validation** : exécution du plan de tests (tests unitaires, tests d'intégration, tests de non-régression), correction des anomalies.
- **Phase 5 – Déploiement et documentation** : mise en production, rédaction du mémoire et des annexes.

5.5 Le pipeline CI/CD

L'intégration continue (CI) et le déploiement continu (CD) constituent une pratique DevOps incontournable dans les projets de développement logiciel modernes. Le pipeline CI/CD mis en place pour IRIS Bank automatise les étapes de validation et de livraison du code, garantissant une qualité constante des livrables.

5.5.1 Intégration Continue (CI)

À chaque *push* ou *pull request* sur le dépôt Git du projet, un workflow automatisé (GitHub Actions) se déclenche et exécute successivement :

- L'installation des dépendances (*npm install*)
- L'exécution des tests automatisés (*npm test*, *Jest*)
- L'analyse statique du code (ESLint) pour détecter les erreurs de style et les bugs potentiels
- La vérification de sécurité des dépendances (*npm audit*)

5.5.2 Déploiement Continu (CD)

Si la phase CI est validée, le pipeline déclenche automatiquement le déploiement de l'application sur l'environnement cible. Ce processus garantit que seul du code testé et validé est mis en production, réduisant considérablement le risque d'introduire des régressions. Dans le cadre pédagogique d'IRIS Bank, cette étape comprend également la mise à jour automatique du schéma de base de données via des scripts de migration.

5.6 Gestion des risques

Toute démarche projet sérieuse intègre une analyse des risques. Plusieurs risques ont été identifiés pour le projet IRIS Bank, avec les stratégies d'atténuation correspondantes :

- **Risque technique – Failles de sécurité** : atténué par l'application systématique des bonnes pratiques OWASP et la réalisation de tests de sécurité dédiés.
- **Risque organisationnel – Retards sur les sprints** : atténué par une estimation rigoureuse lors du sprint planning et l'ajustement du backlog en cas de surcharge.
- **Risque de qualité – Dettes techniques** : atténué par l'intégration des revues de code (code review) et du pipeline CI/CD dès le début du projet.
- **Risque humain – Absentéisme ou départ d'un membre** : atténué par une documentation continue du code et des comptes rendus de réunion partagés.

5.7 Synthèse : adéquation de la démarche avec le projet

La combinaison de la méthode Agile Scrum et du pipeline CI/CD s'est révélée particulièrement adaptée aux contraintes du projet IRIS Bank. La nature évolutive des besoins fonctionnels, la taille réduite de l'équipe et les exigences de qualité et de sécurité ont pleinement justifié ce choix méthodologique.

Cette démarche a permis de maintenir un rythme de développement soutenu tout en garantissant la qualité des livrables, la traçabilité des décisions et la réactivité face aux imprévus. Elle constitue la fondation sur laquelle repose l'ensemble du processus de développement décrit dans les sections suivantes.

6. Expérimentation sur le terrain

6.1 Conception (diagrammes UML)

6.2 Développement

6.3 Plan de tests

7. Recommandations

8. Conclusion

Bibliographie

Barth, A., Jackson, C., & Mitchell, J. C. (2008). Robust defenses for cross-site request forgery. *ACM CCS 2008*.

Beccalli, E., Anolli, M., & Stefanelli, G. (2011). IT and European bank performance. *Journal of Financial Services Research*.

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, UC Irvine.

Gomber, P., Koch, J. A., & Siering, M. (2017). Digital Finance and FinTech. *Journal of Business Economics*, 87(5).

Gray, J., & Reuter, A. (1992). *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann.

McKinsey & Company (2023). *The future of digital banking*. McKinsey Global Institute.

NIST (2020). *Digital Identity Guidelines (SP 800-63B)*. National Institute of Standards and Technology.

OWASP (2021). *OWASP Top Ten 2021*. Open Web Application Security Project.

Richardson, C. (2018). *Microservices Patterns*. Manning Publications.

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. Scrum.org.

Annexes